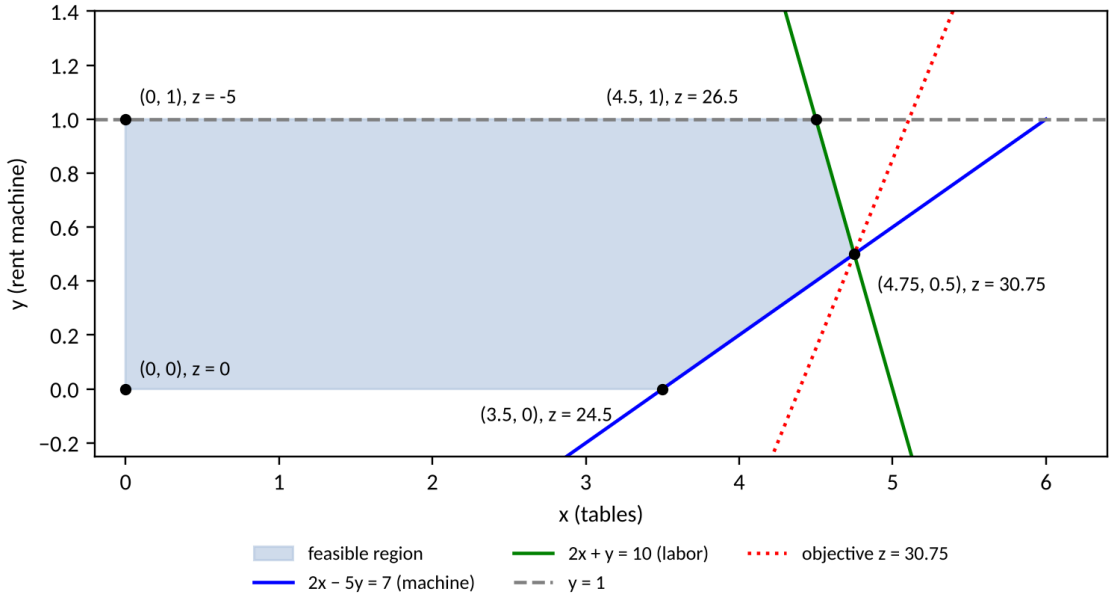
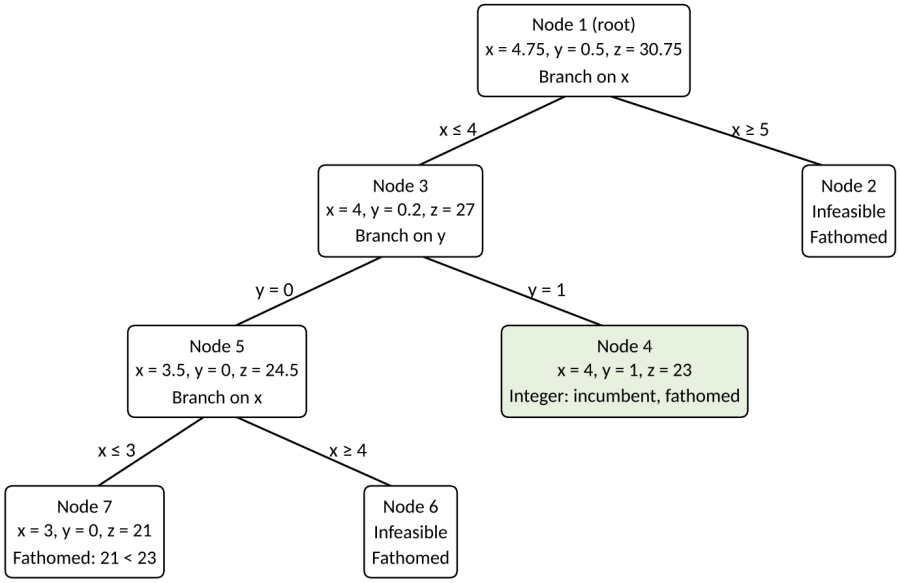
1. (a) The feasible region of the LP relaxation has corner points (0, 0), (3.5, 0), (4.75, 0.5), (4.5, 1), and (0, 1), with objective values 0, 24.5, 30.75, 26.5, and −5. The optimal LP solution is x = 4.75 and y = 0.5, with z = 30.75, where the machine time and labor lines cross.



(b) Rounding to the nearest integers gives (5, 1), which needs 11 hours of labor, and rounding down gives (4, 0), which needs 8 hours of machine time, so neither point is feasible. Rounding an LP solution can leave the feasible region, and even a feasible rounded point may not be optimal, which is why branch and bound is needed.

(c) Each node was solved as an LP in Gurobi. The root branches on x, the x ≥ 5 node is infeasible, and the x ≤ 4 node branches on y. The y = 1 node gives the integer solution x = 4, y = 1, z = 23, which becomes the incumbent. The y = 0 node (z = 24.5) branches on x again, where x ≥ 4 is infeasible and x ≤ 3 (z = 21) cannot beat 23. The optimal solution is to build 4 tables and rent the machine, for z = 23 ($2,300).



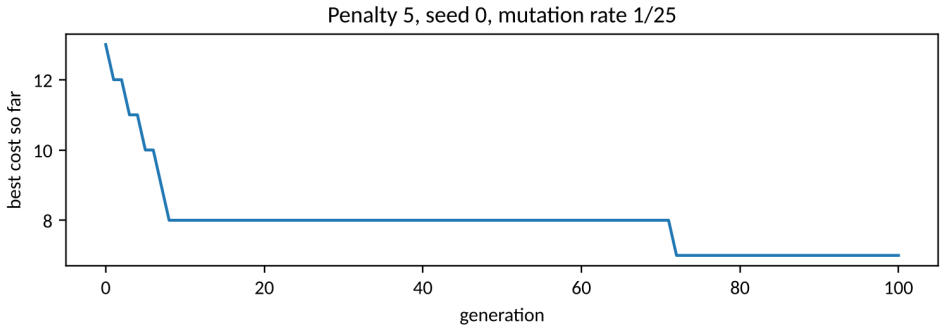
(d) Branching on y fixes it at 0 or 1, which makes the full rent decision, so y is never branched on again below that node. Branching on x only adds a bound that cuts out the fractional values in between, so x can be branched on again further down, as happened at node 5.

2. (a) cost adds the number of towers to the penalty times count_uncovered, and mutate flips each gene when rng.random() is below the rate. Every test value matches:

```
cost, no towers, penalty 5:    125   should be 125
cost, seven towers, penalty 5: 7   should be 7
cost, six towers, penalty 5:   16   should be 16
cost, six towers, penalty 1:   8   should be 8
mutate all towers, rate 0.5: [1, 1, 0, 0, 1, 0, 1, 0]   should start [1, 1, 0, 0, 1, 0, 1, 0]
```

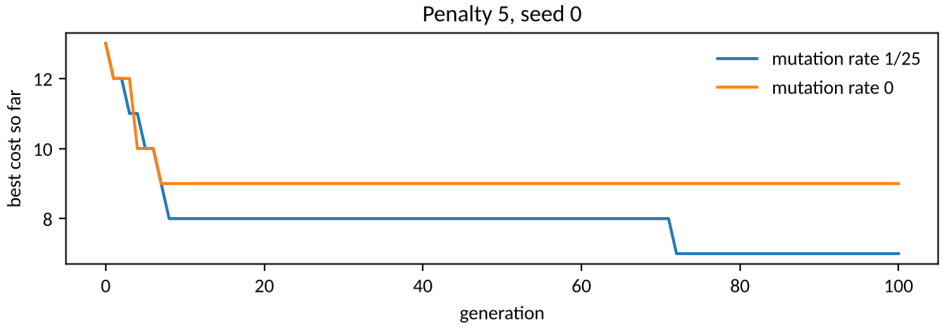
(b) 6 of the 10 runs (seeds 0, 1, 2, 5, 6, and 8) found an optimal solution of 7 towers with no uncovered regions, and the other 4 ended with 8 towers. The genetic algorithm cannot tell on its own that it found an optimal solution, since it has no lower bound and just stops after 100 generations. Only Gurobi's proof in Assignment 2 that no 6 towers work shows 7 is optimal. For seed 0, the best cost reached 8 by generation 8 and 7 at generation 72.

Seed	Towers	Uncovered	Tower regions
0	7	0	2, 5, 7, 14, 16, 20, 23
1	7	0	1, 4, 10, 12, 13, 21, 24
2	7	0	3, 6, 10, 13, 16, 20, 23
3	8	0	4, 5, 6, 7, 14, 17, 22, 25
4	8	0	1, 3, 5, 12, 14, 17, 20, 23
5	7	0	3, 6, 10, 13, 18, 21, 25
6	7	0	1, 4, 9, 12, 19, 21, 24
7	8	0	2, 4, 7, 12, 15, 17, 20, 23
8	7	0	3, 6, 10, 14, 17, 20, 24
9	8	0	3, 5, 6, 12, 14, 18, 21, 25



(c) With no mutation, none of the runs found 7 towers (5 ended with 8 and 5 with 9), and every run stopped improving by generation 12. Crossover only recombines genes already in the population, so once the population fills with copies of a few chromosomes it gets stuck. Mutation adds new genes and keeps the population diverse, which is what let seed 0 improve from 8 to 7 towers at generation 72.

Seed	Towers	Uncovered	Tower regions
0	9	0	3, 5, 6, 8, 15, 17, 18, 20, 25
1	9	0	1, 4, 8, 10, 11, 16, 19, 20, 23
2	8	0	1, 4, 7, 10, 13, 16, 21, 24
3	9	0	1, 3, 8, 10, 11, 13, 19, 22, 24
4	8	0	3, 5, 6, 8, 17, 19, 22, 25
5	9	0	2, 4, 7, 8, 15, 16, 19, 22, 24
6	8	0	1, 4, 8, 9, 11, 19, 22, 24
7	8	0	2, 4, 6, 10, 13, 17, 20, 23
8	9	0	2, 5, 10, 11, 13, 17, 20, 23, 25
9	8	0	2, 4, 6, 10, 12, 13, 21, 24



3. (a) The heuristic places the most constrained shifts first:

1. List the shifts from most conflicts to fewest, breaking ties by the lower shift number.
2. Take the next shift on the list.
3. Assign it to the first hired guard (Guard 1, Guard 2, ...) who has no conflicting shift.
4. If no hired guard can take it, hire a new guard for it.
5. Repeat steps 2 to 4 until every shift is assigned.

Every shift is placed once with exactly one guard, and step 4 means it can always be placed. Step 3 only allows a guard with no conflicting shift, and a new guard has none, so the schedule is always feasible.

(b) Ordering by conflicts gives 5, 2, 3, 4, 6, 7, 1, 8. Shift 5 hires Guard 1 and shift 2 joins it. Shift 3 conflicts with 2 and 5, so it hires Guard 2, and shift 4 joins Guard 2. Shift 6 conflicts with 5 and 4, so it hires Guard 3. Shift 7 joins Guard 2, shift 1 conflicts with 2 and 3 so it joins Guard 3, and shift 8 joins Guard 1. This uses 3 guards: Guard 1 has shifts 2, 5, 8, Guard 2 has 3, 4, 7, and Guard 3 has 1, 6.

(c) Shifts 1, 2, and 3 all conflict, so they need 3 different guards and no schedule can use fewer than 3, which makes this schedule optimal. Since the heuristic is greedy, a bad order can hire extra guards. With shifts a1 to a4 and b1 to b4, where each ai conflicts with every bj except bi, 2 guards are enough, but listing them a1, b1, a2, b2, and so on gives 4. Choosing the next shift as the one whose conflicts already use the most different guards would improve it.